A Sound and Efficient Method for B-Spline Extrema Computation

Thomas Duquette[a,*], André Gallant[a]

[a]*Dynamium Lab, Department of Mechanical Engineering, Université de Moncton, 18 Ave Antonine-Maillet, Moncton, NB, E1A 3E9, Canada*

Abstract

B-splines are widely used in robotic trajectory optimization for manipulators and mobile robots. For such robots to move through the real world, their trajectories must satisfy kinematic constraints that present as bounds actuator position, velocity, acceleration, or higher derivatives. Common approaches either bound the extrema through the control points (convex hull property), which is sound but conservative, or sample the trajectory, which can underestimate the extrema and miss violations. In this work, we propose a new method to find extrema of a B-spline and its derivatives. The method is based on the piecewise-polynomial property of the B-splines, which is combined with a cascading Newton algorithm. We prove that the method is sound and provides strong guarantees under stated assumptions and evaluate it against the convex hull and the discretization methods. Results show the method finds the extrema to a higher precision while maintaining competitive computational efficiency. It can reach relative errors of less than 10^{-7} while having a similar computation time to that of a discretization with 360 points.

Keywords: B-splines, Robot kinematics, Formal verification

1. Introduction

B-splines are widely used in robotic trajectory optimization due to their smoothness, compact parametric representation, and suitability for numerical optimization. These properties make them particularly attractive for generating trajectories for robotic manipulators and autonomous vehicles, where trajectories must satisfy kinematic constraints throughout their execution.

However, verifying these constraints robustly over a continuous trajectory remains challenging. In particular, to establish whether constraints are satisfied at every instant, it is essential to determine the extrema of the B-splines relevant to the prescribed bounds. Exploiting the piecewise-polynomial structure of B-splines makes it possible to formulate extremum finding as a root-finding problem.

Nevertheless, researchers have historically preferred existing approaches that are based on the convex-hull property of B-splines or finite trajectory discretization to assess constraint satisfaction. Although both methods have their benefits, none provides a simulatneously sound and tight approximation of the extrema.

This article addresses the problem of finding extrema of B-spline trajectories by exploiting their piecewise-polynomial representation. We develop a computational pipeline incorporating a cascade of safeguarded Newton searches and establish theoretical guarantees for the proposed method. The resulting approach aims to provide an efficient alternative to conventional trajectory verification techniques without relying on overconservative bounds or dense discretization.

*Corresponding Author

Email addresses: etd9590@umoncton.ca (Thomas Duquette), andre.gallant@umoncton.ca (André Gallant)

1.1. Contributions

The main contributions of this article are:

- We develop a pipeline for finding extrema of B-spline trajectories by exploiting their piecewise-polynomial structure.
- We introduce a cascade of safeguarded Newton searches and formally establish its guarantees.
- We evaluate its accuracy and computational efficiency against the convex hull and the discretization methods.

The article is structured as follows. In Sec. 2, we discuss related works. In Sec. 3, we formally define the problem. We give the proposed method in Sec. 4, and derive its complexity analysis in Sec. 5. We give the experimental setup in Sec. 6, and present and discuss the results in Sec. 7.

2. Related Works

B-splines are widely used to parametrize robot trajectories because they are piecewise polynomial with compact support, which gives local control, and they are C^{p-2} continuous for degree p at knots of multiplicity one, which allows them to be deployed on real hardware [1, 2, 3]. They also satisfy the convex hull property: on each knot span the curve lies in the convex hull of $p+1$ control points. Since the derivative of a B-spline is a B-spline of one degree less whose control points are scaled differences of the original ones, it is possible to derive a sufficient condition for satisfying kinematic limits that is cheap to evaluate and requires no root finding. It is the most common way to enforce kinematic constraints in modern B-spline trajectory optimization, see, e.g. [4, 5]. The bound is not tight, however, and

it can reject trajectories that satisfy the limits, which restricts the performance of the robot in applications where the limits are active [6, 7].

A third approach evaluates the trajectory at a finite set of sample times and takes the extrema of the samples, e.g. in [8, 9, 10]. While generally tighter to the true extremum of the B-spline, this approach is by construction unsound. The approximation may require a dense sampling to approximate the extremum satisfyingly [8]. The sampled extrema therefore never exceed the true ones, so the method can miss a violation of the limits, and it converges to the true value only when the number of samples approaches infinity, at a cost that grows linearly with the number of samples.

The conversion of B-splines to piecewise polynomial form in power form is well established. For instance, Kim et al. use it to simplify computations on B-spline curves [11]. Outside robotics, Fujioka et al. [12] fit B-splines to observational data and develop algorithms to locate the extrema of the spline and its derivatives. Their method is limited to cubic splines.

To the best of our knowledge, no existing method has been shown to simultaneously provide guaranteed enclosures of B-spline extrema with controllable tightness for arbitrary degrees and derivative orders efficiently. This work proposes an approach that aims to achieve these properties.

3. Problem Statement

3.1. Preliminaries: B-spline Trajectories as Piecewise Polynomials

Let $\mathbf{q} : [t_p, t_N] \to \mathbb{R}^J$ be a joint-space trajectory of J joints, represented as a B-spline of degree p with N control points $\mathbf{c}_0, \dots, \mathbf{c}_{N-1} \in \mathbb{R}^J$ and a non-decreasing knot vector $t_0 \le t_1 \le \cdots \le t_{N+p}$:

$$\mathbf{q}(t) = \sum_{n=0}^{N-1} \mathbf{c}_n \, B_{n,p}(t). \tag{1}$$

The basis functions follow the Cox–de Boor recursion,

$$B_{n,0}(t) = \begin{cases} 1 & t \in [t_n, t_{n+1}), \\ 0 & \text{otherwise}, \end{cases} \tag{2}$$

$$B_{n,r}(t) = \frac{t - t_n}{t_{n+r} - t_n} \, B_{n,r-1}(t) + \frac{t_{n+r+1} - t}{t_{n+r+1} - t_{n+1}} \, B_{n+1,r-1}(t), \tag{3}$$

with the convention that terms with a zero denominator vanish. In practice, such degenerate knot spans are empty and can negate the continuity property of the B-spline. Therefore, they are not typically used in trajectory optimization. The basis functions are non-negative, sum to one, and $B_{n,p}$ has compact support $[t_n, t_{n+p+1})$.

Piecewise polynomial structure. Within each of the m knot spans $[t_{i+p}, t_{i+p+1})$, $i = 0, \dots, m-1$ with $m = N - p$, only the $p+1$ basis functions $B_{i,p}, \dots, B_{i+p,p}$ are non-zero, and each is a polynomial of degree p. Hence $\mathbf{q}$ restricted to span i is a vector-valued polynomial depending on the local control points $\mathbf{c}_i, \dots, \mathbf{c}_{i+p}$ only. With the local parameter

$$\tau = \frac{t - t_{i+p}}{\Delta_i} \in [0, 1], \qquad \Delta_i = t_{i+p+1} - t_{i+p}, \tag{4}$$

this polynomial can be written in monomial form as

$$\mathbf{q}(t) = P_i(\tau) = \sum_{j=0}^{p} \mathbf{a}_{i,j} \, \tau^j, \tag{5}$$

with coefficients given by

$$A_i = M_i \, C_i, \tag{6}$$

where $A_i = [\mathbf{a}_{i,0} \ \cdots \ \mathbf{a}_{i,p}]^\top$ and $C_i = [\mathbf{c}_i \ \cdots \ \mathbf{c}_{i+p}]^\top$ both lie in $\mathbb{R}^{(p+1)\times J}$, and the basis matrix $M_i \in \mathbb{R}^{(p+1)\times(p+1)}$ depends only on the knots $t_i, \dots, t_{i+2p+1}$. The polynomial coefficients are therefore a fixed linear map of the control points, which are the decision variables of the trajectory optimization.

Derivatives. For $k \ge 0$, differentiating (5) with respect to time gives

$$\mathbf{q}^{(k)}(t) = \Delta_i^{-k} \, P_i^{(k)}(\tau), \tag{7}$$

where $P_i^{(k)}$ is a polynomial of degree $p - k$ in τ. Note that for uniformly spaced knots, $\Delta_i = T/m$ for total duration T, so the time scaling is the constant factor $(m/T)^k$. Since $B_{n,p} \in C^{p-2}$ at knots of multiplicity one, $\mathbf{q}$ and its derivatives up to order $p-2$ are continuous across span boundaries, while $\mathbf{q}^{(p)}$ is piecewise constant and generally discontinuous. In particular, $P_i^{(p-2)}$ is linear in τ on each span, a fact exploited in Section 4.

3.2. Problem Statement

Let $\mathbf{q} : [0, T] \to \mathbb{R}^J$ be a joint-space trajectory represented by a B-spline of degree p with N control points and total duration T, as defined in Section 3.1, and let q_j denote its j-th joint component. Kinematic limits on velocity, acceleration, jerk, and higher-order derivatives are constraints of the form

$$\underline{L}_{j,k} \ \le \ q_j^{(k)}(t) \ \le \ \overline{L}_{j,k} \qquad \forall \, t \in [0, T], \tag{8}$$

for joints $j = 1, \dots, J$ and derivative orders $k = 0, \dots, d$. Verifying (8) continuously requires the extrema of each derivative over the entire continuous time domain,

$$q_{j,k}^{\min} = \min_{t \in [0,T]} q_j^{(k)}(t), \qquad q_{j,k}^{\max} = \max_{t \in [0,T]} q_j^{(k)}(t), \tag{9}$$

Problem 1 (Certified derivative extrema)**.** *Given the control points $\mathbf{c}_0, \dots, \mathbf{c}_{N-1}$, the degree p, the knot vector, the duration T, a maximal derivative order d, and a tolerance $\varepsilon > 0$, compute for every joint j and every order $k = 0, \dots, d$ an enclosure $[\,\underline{b}_{j,k}, \overline{b}_{j,k}\,]$ such that:*

1. (Soundness) $\underline{b}_{j,k} \le q_{j,k}^{\min}$ *and* $q_{j,k}^{\max} \le \overline{b}_{j,k}$*;*
2. (Tightness) *the overestimation $q_{j,k}^{\min} - \underline{b}_{j,k}$ and $\overline{b}_{j,k} - q_{j,k}^{\max}$ is controlled by ε and vanishes as $\varepsilon \to 0$.*

Soundness guarantees that no violation of (8) is missed: if $\underline{L}_{j,k} \le \underline{b}_{j,k}$ and $\overline{b}_{j,k} \le \overline{L}_{j,k}$, the trajectory is certified feasible for all $t \in [0, T]$. Tightness limits false rejections of feasible trajectories. As $\mathbf{q}^{(k)}$ is piecewise polynomial of degree $p - k$ with $p - k \ge 2$ for $k \le d$, the problem reduces to finding, on each of the $m = N - p$ segments, the real roots of the polynomial $P_i^{(k+1)}$ of degree $p - k - 1$ in $[0, 1]$, which has no closed-form solution in general for $p - k - 1 \ge 5$.

4. Proposed method

We propose to start by segmenting the B-spline in its polynomials using classical methods described in [11], and proceed by finding the roots of each polynomial sequentially, starting at the piecewise linear polynomial (which has derivative order $k = p - 2$), and cascading to higher derivative orders using the procedure described in Algorithm. 1.

4.1. Soundness of the Cascade

We now prove the soundness of the cascade algorithm, supported by three assumptions.

Notation. Fix one joint and one segment, and write P for its local polynomial on $[0, 1]$ and $f_k = P^{(k)}$ for its k-th derivative, a polynomial of degree $p - k$. Let $Z_k = \{\tau \in [0, 1] : f_k(\tau) = 0\}$ denote its real roots in $[0, 1]$. A finite set $\mathcal{R}$ of closed intervals contained in $[0, 1]$ *encloses* a set $Z \subseteq [0, 1]$ if $Z \subseteq \bigcup \mathcal{R}$. Point intervals $[a, a]$ are allowed. Enclosure is a statement about coverage only: it carries no information on how many roots lie in each interval. We write $\mathbf{f}(X)$ for the interval evaluation of f over an interval X.

Assumption 1 (Interval evaluation)**.** *For every polynomial f and interval $X \subseteq [0, 1]$, $f(\tau) \in \mathbf{f}(X)$ for all $\tau \in X$.*

Assumption 2 (Sign tests)**.** *The signs of f_k at the points used in* Refine *and* SafeNewton*, and of products of such values, are evaluated correctly.*

Assumption 3 (Base case)**.** *The initial set $\mathcal{R}_{p-2}$ in Algorithm 1 encloses Z_{p-2}. Since f_{p-2} is linear, this holds with the point interval $[-a_0/a_1, -a_0/a_1]$ when $f_{p-2}(\tau) = a_1\tau + a_0$ with $a_1 \ne 0$ and the root lies in $[0, 1]$. If $a_1 = 0 \ne a_0$ then $Z_{p-2} = \emptyset$ and $\mathcal{R}_{p-2} = \emptyset$. If $f_{p-2} \equiv 0$ we take $\mathcal{R}_{p-2} = \{[0, 1]\}$.*

Lemma 1 (Reduction to segments)**.** *Let $0 \le k \le p - 2$, let the interior knots have multiplicity one, and let $\Delta_i = t_{i+p+1} - t_{i+p}$. Then the range of $q_j^{(k)}$ over $[t_p, t_N]$ equals the union over segments i of the ranges of $\Delta_i^{-k} P_{i,j}^{(k)}$ over $[0, 1]$.*

Proof. On the half-open span $[t_{i+p}, t_{i+p+1})$ we have $q_j^{(k)}(t) = \Delta_i^{-k} P_{i,j}^{(k)}(\tau)$ with $\tau \in [0, 1)$. At an interior knot, $q_j^{(k)}$ is continuous because $q_j \in C^{p-2}$, so its value is the left limit $\Delta_i^{-k} P_{i,j}^{(k)}(1)$. At t_N the value is defined by the same continuity. Hence every value of $q_j^{(k)}$ is attained by some piece on $[0, 1]$, and every value of every piece on the closed interval $[0, 1]$, including $\tau = 1$, is attained by $q_j^{(k)}$. □

Algorithm 1 Certified extrema of a B-spline and its derivatives

Require: Segment polynomials $P_1, \dots, P_m$ of degree p on $[0, 1]$, order $d < p - 2$, tolerance ε, duration T

Ensure: Interval enclosures $\mathcal{E}_k$ of all candidate extrema of $P^{(k)}$, $k = 0, \dots, d$

procedure SafeNewton(f, ℓ, u, ε) ▷ requires $f(\ell)f(u) \le 0$; returns $[\ell, u]$ with $u - \ell < \varepsilon$ containing a root
  $x \leftarrow (\ell + u)/2$
  while true **do**
    if $f(\ell)\, f(x) \le 0$ **then** $u \leftarrow x$
    else $\ell \leftarrow x$
    end if ▷ shrink bracket
    if $u - \ell < \varepsilon$ **then**
      return $[\ell, u]$
    end if
    $x_N \leftarrow x - f(x)/f'(x)$
    $x \leftarrow x_N$ if $\ell < x_N < u$, **else** $(\ell + u)/2$ ▷ bisection fallback
  end while
end procedure

procedure Refine($f, \mathcal{R}$) ▷ $\mathcal{R}$ encloses the roots of f'
  $\mathcal{R}' \leftarrow \{\, r \in \mathcal{R} : 0 \in \mathbf{f}(r) \,\}$ ▷ keep enclosures that may hold a root
  for each gap $[g, h]$ of $[0, 1] \setminus \bigcup \mathcal{R}$ **do** ▷ f monotone on $[g, h]$
    if $f(g)\, f(h) \le 0$ **then**
      $\mathcal{R}' \leftarrow \mathcal{R}' \cup \{\text{SafeNewton}(f, g, h, \varepsilon)\}$
    end if
  end for
  return $\mathcal{R}'$
end procedure

$\mathcal{E}_k \leftarrow \emptyset$ for $k = 0, \dots, d$
for $i = 1, \dots, m$ **do**
  $\mathcal{R} \leftarrow \{\tau \in [0, 1] : P_i^{(p-2)}(\tau) = 0\}$ as point intervals ▷ linear: exact
  for $k = p - 2, \dots, 0$ **do**
    if $k \le d$ **then**
      $\mathcal{E}_k \leftarrow \mathcal{E}_k \cup \{\mathbf{P_i^{(k)}}(r) : r \in \mathcal{R}\} \cup \{P_i^{(k)}(0),\ P_i^{(k)}(1)\}$
    end if
    if $k > 0$ **then**
      $\mathcal{R} \leftarrow \text{Refine}(P_i^{(k)}, \mathcal{R})$
    end if
  end for
end for
return $(m/T)^k\, \mathcal{E}_k$ for $k = 0, \dots, d$ ▷ time normalization

Lemma 2 (Candidate set)**.** *Let f be a polynomial and let $\mathcal{R}$ enclose the roots of f' in $[0,1]$. Define the candidate set*

$$C(f,\mathcal{R}) = \{\,[f(0),f(0)],\ [f(1),f(1)]\,\} \cup \{\,\mathbf{f}(r) : r \in \mathcal{R}\,\}. \quad (10)$$

Then $\min_{[0,1]} f$ and $\max_{[0,1]} f$ both belong to $\bigcup C(f,\mathcal{R})$. Consequently, $f(\tau) \in [\inf \bigcup C,\ \sup \bigcup C]$ for all $\tau \in [0,1]$.

Proof. f is continuous on the compact set $[0,1]$, so it attains its minimum and maximum at some $\tau^\star$. If $\tau^\star \in \{0,1\}$, then $f(\tau^\star)$ is one of the first two members of C. Otherwise $\tau^\star \in (0,1)$ is an interior extremum of a differentiable function, so $f'(\tau^\star) = 0$ by Fermat's theorem. Then $\tau^\star$ lies in some $r \in \mathcal{R}$, and $f(\tau^\star) \in \mathbf{f}(r)$ by Assumption 1. The last claim follows because every value of f lies between its minimum and maximum. □

Lemma 3 (SafeNewton)**.** *Let f be continuous with $f(\ell_0)f(u_0) \le 0$. Then, under Assumption 2, every interval $[\ell,u]$ held by Algorithm* **??** *after any update step satisfies $[\ell,u] \subseteq [\ell_0,u_0]$ and $f(\ell)f(u) \le 0$. In particular, whenever the algorithm returns, $[\ell,u]$ contains a root of f in $[\ell_0,u_0]$. This holds regardless of how the iteration x is chosen, as long as $x \in [\ell,u]$, and independently of ε and of any iteration cap.*

Proof. We argue by induction on the updates. The invariant holds initially. In one update, either $f(\ell)f(x) \le 0$ and we set $u \leftarrow x$, which gives $f(\ell)f(u) = f(\ell)f(x) \le 0$ directly. Or $f(\ell)f(x) > 0$, so $f(\ell) \ne 0$ and $f(x)$ has the same sign as $f(\ell)$, and we set $\ell \leftarrow x$. The point x is either a Newton iterate accepted only if $\ell < x_N < u$, or the midpoint of $[\ell,u]$, so $x \in [\ell,u]$ and the bracket stays inside $[\ell_0,u_0]$. Finally, $f(\ell)f(u) \le 0$ and continuity give a root in $[\ell,u]$ by the intermediate value theorem. □

Lemma 4 (Refine)**.** *Let f be a polynomial and let $\mathcal{R}$ be a finite set of closed intervals in $[0,1]$ enclosing the roots of f' in $[0,1]$. Then, under Assumptions 1 and 2, $\mathcal{R}' = \textsc{Refine}(f,\mathcal{R})$ is a finite set of closed intervals in $[0,1]$ enclosing the roots of f in $[0,1]$.*

Proof. Finiteness and the inclusion in $[0,1]$ hold because $\mathcal{R}'$ consists of members of $\mathcal{R}$ and of intervals returned by SafeNewton on subintervals of $[0,1]$. Let $U = \bigcup \mathcal{R}$ and let $\tau_0 \in [0,1]$ satisfy $f(\tau_0) = 0$.

Case 1: $\tau_0 \in U$. Then $\tau_0 \in r$ for some $r \in \mathcal{R}$, and $0 = f(\tau_0) \in \mathbf{f}(r)$ by Assumption 1. So r passes the test $0 \in \mathbf{f}(r)$ and is kept in $\mathcal{R}'$.

Case 2: $\tau_0 \notin U$. The set U is closed, so $[0,1] \setminus U$ is relatively open in $[0,1]$ and is a finite union of disjoint intervals. Let Γ be the component containing τ_0 and $[g,h]$ its closure, which equals the gap examined by Refine, with $g < h$. Since $\Gamma \cap U = \emptyset$, f' has no root in the interior (g,h). By continuity f' has constant strict sign there, so f is strictly monotone on $[g,h]$ by the mean value theorem. As $f(\tau_0) = 0$ with $\tau_0 \in [g,h]$, strict monotonicity gives that $f(g)$ and $f(h)$ do not have the same strict sign, so $f(g)f(h) \le 0$. Hence the test of Refine passes and $\textsc{SafeNewton}(f,g,h,\varepsilon)$ is called. By Lemma 3 it returns an interval containing a root of f in $[g,h]$. Strict monotonicity on $[g,h]$ means τ_0 is the only such root, so the returned interval contains τ_0.

In both cases $\tau_0 \in \bigcup \mathcal{R}'$. □

Theorem 1 (Soundness of the cascade)**.** *Let Assumptions 1–3 hold and let $0 \le d \le p-2$. For each joint j, derivative order $k \le d$ and segment i, let $\mathcal{R}_k^{(i)}$ be the set $\mathcal{R}$ at the start of iteration k of Algorithm 1, and let $\mathcal{E}_k$ be the output for order k. Then:*

1. *$\mathcal{R}_k^{(i)}$ encloses Z_{k+1} for every $k \in \{p-2,\dots,0\}$;*
2. *for every $t \in [t_p,t_N]$, $q_j^{(k)}(t) \in [\inf \bigcup \mathcal{E}_k,\ \sup \bigcup \mathcal{E}_k]$, in particular $\underline{b}_{j,k} \le q_{j,k}^{\min}$ and $q_{j,k}^{\max} \le \overline{b}_{j,k}$, with $\underline{b}_{j,k} = \inf \bigcup \mathcal{E}_k$ and $\overline{b}_{j,k} = \sup \bigcup \mathcal{E}_k$.*

Proof. Fix a segment and a joint, and drop the indices i, j.

(1) Invariant, by downward induction on k. For $k = p-2$, the set $\mathcal{R}$ is the set from Assumption 3, which encloses $Z_{p-2} = Z_{k+1}$. Suppose $\mathcal{R}_k$ encloses Z_{k+1} for some $k \ge 1$. Algorithm 1 sets $\mathcal{R}_{k-1} = \textsc{Refine}(f_k,\mathcal{R}_k)$. Since $f_k' = f_{k+1}$, Lemma 4 applies and shows that $\mathcal{R}_{k-1}$ encloses $Z_k = Z_{(k-1)+1}$.

(2) Segment bounds. For $k \le d$, the algorithm adds to $\mathcal{E}_k$ the set $C(f_k,\mathcal{R}_k)$ multiplied by the segment-constant factor $(\Delta_i)^{-}k$. Since $\mathcal{R}_k$ encloses the roots of $f_k' = f_{k+1}$ by (1), Lemma 2 gives that $f_k(\tau) \in [\inf \bigcup C, \sup \bigcup C]$ for all $\tau \in [0,1]$.

(3) Combination. By Lemma 1, every value of $q_j^{(k)}$ on $[t_p,t_N]$ is a value of $\Delta^{-k}P_{i,j}^{(k)}$ on $[0,1]$ for some segment i. By (2) it lies in the interval spanned by the candidates of that segment, and hence in the interval spanned by the union over all segments, which is $[\inf \bigcup \mathcal{E}_k, \sup \bigcup \mathcal{E}_k]$. □

Remark 1 (What is used about retained intervals)**.** *The proof only uses coverage. An interval retained in* Refine *because $0 \in \mathbf{f}(r)$ may contain zero, one, or several roots of f. Passing it unchanged to the next level is sound, and the rare event in which this happens affects computation time but not the validity of the result.*

Remark 2 (Role of ε and of the iteration cap)**.** *By Lemma 3, soundness does not depend on ε nor on whether* SafeNewton *exits at the width test or at an iteration cap. Tightness and running time do depend on them and are not covered by Theorem 1.*

Remark 3 (Floating-point arithmetic)**.** *The proof is stated for exact sign evaluation and an outward-rounded interval evaluator. In an implementation, Assumption 2 holds if the sign tests in* Refine *and* SafeNewton *are certified, for instance by deciding the sign of $f(x)$ from the interval $\mathbf{f}(x)$ and treating undecided cases as sign changes, and if the base root $-a_0/a_1$ is replaced by its outward-rounded enclosure.*

5. Complexity analysis

Notation. We consider a clamped, uniform B-spline for the complexity analysis and the experiments. The trajectory has N control points, degree p, J joints, and $m = N - p$ segments. The local tolerance is $\varepsilon_\ell = m\,\varepsilon$. At cascade level k the polynomial $f_k = P^{(k)}$ has degree $\delta = p - k \in \{2,\dots,p\}$. We count real or interval arithmetic operations, and an interval operation costs a constant factor more than a real one. Evaluating a polynomial of degree δ by Horner's scheme, or its derivative, costs $O(\delta)$. Let n_k be the number of root enclosures held at level k, and let $r = \max_k n_k$ be the largest such number. Let I be the maximal number of iterations of one call to SafeNewton.

Proposition 1 (Time complexity)**.** *Algorithm 1 runs in*

$$T(N, p, J) = O(J\,(N-p)\,r\,I\,p^2) \tag{11}$$

operations, and since $r \le p-2$ in the generic case below, $T = O(J\,(N-p)\,I\,p^3)$.

Proof. The segments and joints are processed independently, so the total cost is $J\,m$ times the cost for one polynomial P of degree p. We count it step by step.

Local polynomial. The coefficients of the segment are obtained as $A_i = M_i C_i$ with $M_i \in \mathbb{R}^{(p+1)\times(p+1)}$ and $C_i \in \mathbb{R}^{(p+1)\times J}$, which costs $O(p^2 J)$ per segment, i.e., $O(p^2)$ per polynomial.

Derivatives. Computing $f_1, \dots, f_{p-2}$ by repeated differentiation of the coefficient array costs $\sum_{\delta=2}^{p} O(\delta) = O(p^2)$.

Cascade level k ($\delta = p-k$). With $n_k \le r$ root enclosures at hand, Refine performs

1. n_k interval evaluations of f_k for the retention test, and the same number again for recording candidates when $k \le d$, i.e., $O(n_k \delta)$;
2. $O(n_k + 1)$ point evaluations at the gap boundaries, i.e., $O((n_k+1)\delta)$;
3. at most $n_k + 1$ calls of SafeNewton, one per gap. Each call forms f_k' in $O(\delta)$ and performs at most I iterations, each with two Horner evaluations, which costs $O(I\delta)$.

The Newton calls dominate, so the cost of level k is $O((n_k + 1)\,I\,\delta) = O(r\,I\,\delta)$.

Sum over levels. $\sum_{\delta=2}^{p} O(rI\delta) = O(r\,I\,p^2)$. Adding the $O(p^2)$ terms above gives $O(rIp^2)$ per polynomial and $O(J\,m\,r\,I\,p^2)$ in total. Finally, $n_k \le \delta - 1$ when each enclosure contains at most one real root of f_{k+1}, which gives $r \le p-2$. □

Remark 4 (Dependence on N, p, J)**.** *For a fixed degree, the cost is linear in the number of control points and in the number of joints. In p it is quadratic when the number of root enclosures per level is bounded by a constant (the typical case), and cubic in the worst case $r = p-2$. The segments and joints are independent and can be processed in parallel. The cost does not decrease with a smaller d, since the roots of $f_1, \dots, f_{d+1}$ are needed to report the extrema of all orders $0, \dots, d$. Only the candidate recording of order $k \le d$ depends on d, with a cost of $O(\delta\, n_k)$ per level, already included above.*

Remark 5 (Retained intervals)**.** *The bound $r \le p-2$ assumes that every enclosure contains at most one root. An interval retained because $0 \in \mathbf{f}(r)$ may contain no root, which can increase n_k by at most the number of new real roots found at that level, so in general $n_k \le n_{k+1} + (p-k)$ and $r = O(p^2)$. The per-level cost is then $O(p^3)$ and the total per polynomial $O(p^4)$. Retention was never triggered in our experiments, so the bound of Proposition 1 applies in practice.*

6. Experimental Setup

We compare the proposed method against two baselines that also solve Problem 1: a discretization-based method, which evaluates $q_j^{(k)}$ at multiple values of n_s uniformly spaced samples and takes the minimum and maximum of the samples, a convex-hull-based method, which bounds $q_j^{(k)}$ by the range of the control points of the derivative B-spline, and a variant of the proposed method in which the Newton cascade is replaced by computing the real roots of f_{k+1} in $[0, 1]$ as eigenvalues of its companion matrix, for each $k \le d$.

The variant uses Eigen's c++ implementation for the companion matrix method [13, 14], and evaluates f_k at these roots and at the segment endpoints, using the same piecewise polynomial conversion. Hence, the comparison between the variant and the convex-hull and discretization methods isolates the effect of the polynomial decomposition, and the comparison between the cascade and the variant isolates the effect of the Newton cascade as the root-finding step. The companion variant does not enclose the roots, so its bounds are not certified and $\rho < 1$ is possible. All methods are applied to derivative orders $k = 0, \dots, d$ with $d = 2$, i.e. searching for extrema of position, velocity, and acceleration.

The evaluation uses two datasets of uniform, clamped B-splines (Table 1), chosen to represent B-splines typically used in trajectory optimization. The proposed method is not restricted to this setting: it applies to any B-spline with interior knots of multiplicity one and $d \le p-2$. The randomized dataset contains 5 600 B-splines with $N \in \{12, 14, \dots, 26\}$ control points and degrees $p \in \{4, 5, \dots, 10\}$. The optimized dataset contains 25 600 B-splines generated by trajectory optimization with blast [10], all with $N = 16$ control points and degree $p = 5$. Every B-spline is defined for a UR5e manipulator with $J = 6$ joints, and each method is applied to every joint independently to bound the extrema $q_{j,k}^{\min}$ and $q_{j,k}^{\max}$ of Problem 1.

Table 1: Datasets of uniform, clamped B-splines for a UR5e manipulator ($J = 6$). N: number of control points, p: degree.

Dataset	B-splines	N	p	Source
Randomized	5 600	12, 14, …, 26	4–10	Random
Optimized	25 600	16	5	blast

Reference and error metric. Ground truth is obtained by a fine discretization: for every joint j and derivative order $k \in \{0, 1, 2\}$ (position, velocity, acceleration), $q_j^{(k)}$ is evaluated at 100 000 uniformly spaced samples over the trajectory duration, and their minimum and maximum values give the reference bounds $\underline{b}_{j,k}^{\mathrm{ref}}$ and $\overline{b}_{j,k}^{\mathrm{ref}}$. Because the samples are a subset of the trajectory, these bounds lie within the true extrema. To compare methods independently of the units and limits of each joint, each pair of bounds $[\underline{b}, \overline{b}]$ is converted into the normalized constraint value

$$u_{j,k} = \frac{2}{w_{j,k}} \max\{\, |\underline{b} - c_{j,k}|,\ |\overline{b} - c_{j,k}| \,\}, \tag{12}$$

with the limit center $c_{j,k} = (\overline{L}_{j,k} + \underline{L}_{j,k})/2$ and width $w_{j,k} = \overline{L}_{j,k} - \underline{L}_{j,k}$, where $[\underline{L}_{j,k}, \overline{L}_{j,k}]$ are the limits of (8).

Here $u_{j,k} \le 1$ means that the limits are respected and $u_{j,k} > 1$ that they are violated. A trajectory is labelled infeasible if $u_{j,k}^{\mathrm{ref}} >$

1 for some j and k. The error of a method $\mathcal{M}$ is reported as the fraction

$$\rho_{j,k}^{\mathcal{M}} = \frac{u_{j,k}^{\mathcal{M}}}{u_{j,k}^{\mathrm{ref}}}, \tag{13}$$

so that $\rho = 1$ means agreement with the reference, $\rho < 1$ means that the extrema are underestimated, and $\rho > 1$ means that they are overestimated. A sound method returns bounds that contain the true extrema, hence $\rho \geq 1$ for every j and k. A violation is missed when $u_{j,k}^{\mathcal{M}} \leq 1 < u_{j,k}^{\mathrm{ref}}$.

The benchmarks were conducted on a computer equipped with an AMD Ryzen 9 9950X 16-Core Processor and 64GB of RAM. The code is written in C++ and an algorithm's benchmark is ran 15 times, discarding the first five runs to minimize CPU warm-up effects. Note that, since all algorithms in this benchmark are deterministic, the multiple runs only serve to reduce the variability of the computation time.

7. Results and discussion

We first study the influence of the tolerance ε on accuracy and computation time (Fig. 1). The tolerance bounds the width of every root enclosure returned by SafeNewton, measured as a fraction of the normalized trajectory duration, so that the local tolerance on each segment is $m\,\varepsilon$ with $m = N - p$. We sweep $\varepsilon \in \{10^{-}1, 10^{-}7\}$ on the combined dataset, i.e., the union of the randomized and optimized sets of Table 1 (31 200 B-splines), with derivative orders $k = 0, \ldots, d$ and $d = 2$. Note that separating the datasets does not influence the results beyond being generally faster for the optimized trajectories, as they tend to be smoother and have less extrema, thus reducing the number of Newton calls. By Theorem 1, the returned bounds contain the true extrema for every ε, so $\rho \geq 1$ throughout. Therefore ε affects only the tightness of the enclosures and the computation time.

Decreasing ε monotonically reduces the overestimation $\rho - 1$, from 70% at $\varepsilon = 10^{-1}$ to $< 10^{-}9\%$ at $\varepsilon = 10^{-7}$, while the computation time grows by a factor of only 16.8% over the same range. This weak dependence is consistent with the convergence of Newton's method near simple roots: each additional decade of accuracy costs few extra iterations, whereas a bisection-only search would need $\log_2 10 \approx 3.3$ more iterations per decade. Between $\varepsilon = 10^{-1}$ and $\varepsilon = 10^{-2}$ the computation time decreases instead. A plausible cause is that, for large ε, root enclosures remain wide enough for the interval evaluation of the next derivative to contain zero, so they are retained rather than resolved. This increases the number of enclosures per level and with it the number of interval and point evaluations (Section 5).

We then compare the proposed method with $\varepsilon = 10^{-7}$ against three alternatives on the same dataset and metrics (Fig. 2): the convex-hull method, the discretization method with n_s uniformly spaced samples, and the companion matrix variant. The convex-hull, the Newton cascade and companion variants do not depend on n_s, which is the horizontal axis of Fig. 2 and appears there as a constant line.

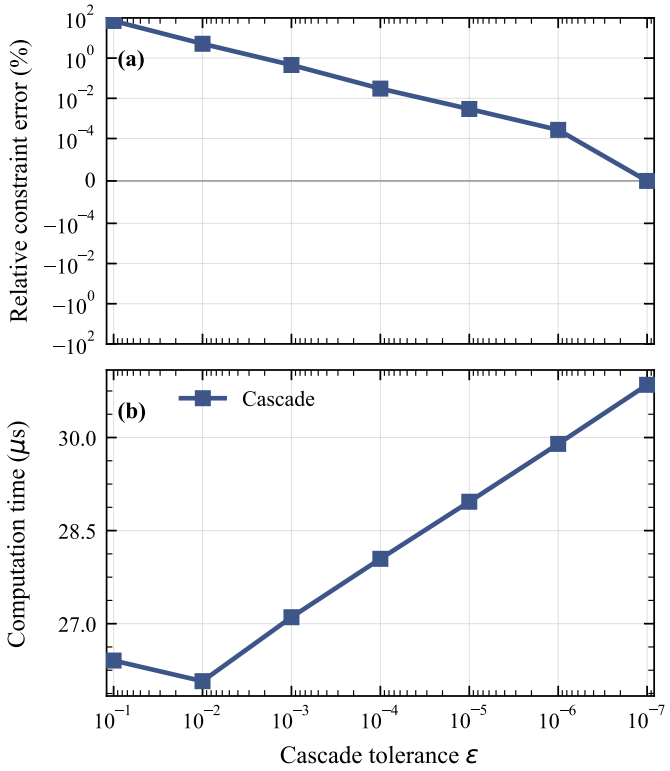


Figure 1: Ablation on the tolerance ε on the combined dataset. Top: mean relative overestimation $\rho - 1$ with respect to the reference. Bottom: mean computation time per B-spline. Shaded regions show XX. The horizontal axis and error axis are logarithmic.

Fig. 2 shows that neither baseline satisfies both requirements of Problem 1. The convex-hull method is the fastest, 79.3× faster than the proposed method, but it violates tightness: its mean overestimation is 83.4% ($\rho \approx 1.834$), and it rejects XX% of the trajectories that are feasible according to the reference. Its bounds depend only on the control points of the derivative spline and not on the extrema actually attained, so they are conservative by construction. The discretization method violates soundness. At n_s = XX it misses XX% of the trajectories for which the reference is violated. The missed-violation rate decreases with n_s, but no finite n_s guarantees soundness, and a safety margin that compensates for the underestimation cannot be certified without knowing the extrema.

The proposed method and the companion variant both reach a mean overestimation below 10^{-7}. They differ in their guarantees. The proposed method evaluates f_k by interval arithmetic over each root enclosure, so its bounds account for the remaining uncertainty on the root location and, under the assumptions of Theorem 1, contain the true extrema. The companion variant evaluates f_k at approximate roots obtained from an eigenvalue problem, whose accuracy degrades for ill-conditioned polynomials, in particular with multiple or clustered roots, and it may then yield $\rho < 1$. We did not observe this: the minimum ρ over the dataset is XX. The proposed method is 2.4× faster than the companion variant in mean computation time at $\varepsilon = 10^{-7}$. Its computation time matches that of the discretization method at

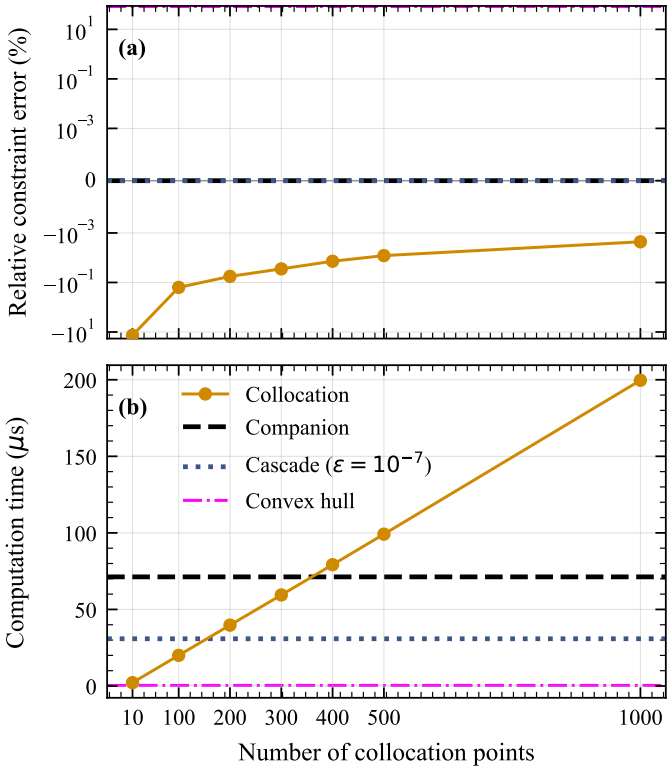


Figure 2: Comparison of the methods on the combined dataset. Top: mean relative overestimation $\rho - 1$ with respect to the reference. Bottom: mean computation time per B-spline. The horizontal axis is the number of samples n_s of the discretization method. All other methods are independent of n_s and are drawn as constant lines.

$n_s \approx 360$, where the latter has a mean $\rho - 1$ of XX and misses XX% of the violations.

8. Conclusions and future work

We presented a method that computes sound and tight enclosures of the extrema of a B-spline and of its derivatives up to order $d \leq p-2$ over the whole trajectory, rather than at sampled times. On 31 200 uniform, clamped B-splines for a UR5e, the method with $\varepsilon = 10^{-7}$ has a mean overestimation below 10^{-7} relative to a 100 000-point discretization reference and missed no violation of that reference. The convex-hull method is faster but overestimates the extrema by 83.4% on average, and the discretization method underestimates them and missed XX% of the violations at $n_s = $ XX. Comparing the cascade with a variant using companion-matrix roots separates the two components of the method. The piecewise polynomial decomposition accounts for the accuracy gain over the convex-hull and discretization methods, as both root-finding variants share it. The Newton cascade accounts for a 2.4× reduction in computation time over the companion variant, and, unlike the companion variant, its bounds account for the uncertainty on the root locations.

The soundness guarantee holds for the stated class of B-splines (interior knots of multiplicity one, $d \leq p - 2$) and relies on certified interval evaluation and sign tests; extending these guarantees to a fully rounding-aware implementation is a necessary step towards its use in safety-critical settings. Further work includes extending the method to other parametric curves such as NURBS, and using the algorithm inside the trajectory optimization loop instead of only as a final certification step.

8.1. CRediT author statement

Thomas Duquette: **André Gallant**: Resources, Writing - Review and Editing, Supervision, Funding Acquisition.

8.2. Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

8.3. Acknowledgments

Funding: The sponsors' contribution is strictly financial, and they had no research role in the present project. The authors would like to thank Nikos Doiron for proofreading and commenting, contributing to the quality of the paper.

8.4. Data availability

Data will be made available on request.

References

[1] A. Gasparetto, V. Zanotto, A new method for smooth trajectory planning of robot manipulators, Mechanism and Machine Theory 42 (2007) 455–471. doi:10.1016/j.mechmachtheory.2006.04.002.

[2] H. Liu, X. Lai, W. Wu, Time-optimal and jerk-continuous trajectory planning for robot manipulators with kinematic constraints, Robotics and Computer-Integrated Manufacturing 29 (2013) 309–317. doi:10.1016/j.rcim.2012.08.002.

[3] S. Lu, J. Zhao, L. Jiang, H. Liu, Solving the time-jerk optimal trajectory planning problem of a robot using augmented lagrange constrained particle swarm optimization, Wiley Online Library 2017 (2017). doi:10.1155/2017/1921479.

[4] Y. Zhou, G. Han, Z. Wei, Z. Huang, X. C. A. Sciences, undefined 2023, A time-optimal continuous jerk trajectory planning algorithm for manipulators, mdpi.comY Zhou, G Han, Z Wei, Z Huang, X ChenApplied Sciences, 2023•mdpi.com.

[5] C. Wang, T. Xu, M. Tomizuka, Continuous trajectory optimization via b-splines for multi-jointed robotic systems (12 2022).

[6] C. De Boor, J. W. Daniel, Splines with nonnegative b-spline coefficients, Mathematics of computation 28 (126) (1974) 565–568.

[7] W. V. Loock, G. Pipeleers, J. S. M. Sciences, undefined 2015, B-spline parameterized optimal motion trajectories for robotic systems with guaranteed constraint satisfaction, ms.copernicus.orgW Van Loock, G Pipeleers, J SweversMechanical Sciences, 2015•ms.copernicus.org 6 (2015) 163–171. doi:10.5194/ms-6-163-2015.

[8] M. Paing, N. U. T. I. J. of Advanced, undefined 2022, A spline-based approach to smooth and time-optimal trajectory generation for cnc machines with guaranteed kinematic constraints, SpringerMS Paing, N UchiyamaThe International Journal of Advanced Manufacturing Technology, 2022•Springer 121 (2022) 3385–3398. doi:10.1007/S00170-022-09517-1.

[9] A. Gallant, C. Gosselin, Extending the capabilities of robotic manipulators using trajectory optimization, Mechanism and Machine Theory 121 (2018) 502–514. doi:10.1016/j.mechmachtheory.2017.09.016.

[10] N. Doiron, T. Duquette, G. V. T. Djogdom, A. Gallant, Accelerating trajectory optimization by exploiting b-spline gradient structure, in: 2026 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2026, pp. 22372–22379.

[11] D. S. Kim, J. Ryu, H. C. Lee, H. Shin, The conversion of a dynamic b-spline curve into piecewise polynomials in power form, Computer-Aided Design 34 (2002) 337–345. doi:10.1016/S0010-4485(01)00090-2.

[12] H. Fujioka, H. Kano, Optimal smoothing splines for detecting extrema from observational data, in: Proceedings of the 44th IEEE Conference on Decision and Control, IEEE, 2005, pp. 3717–3722.

[13] A. Edelman, H. Murakami, Polynomial roots from companion matrix eigenvalues, Mathematics of Computation 64 (210) (1995) 763–776.

[14] G. Guennebaud, B. Jacob, et al., Eigen, https://libeigen.gitlab.io (2010).